The Alien Tiles Problem

CSPLib Problem 027 — Problem Description,
ILP and SAT Models, and Datasets

March 14, 2026

Abstract

The *Alien Tiles* puzzle is a combinatorial puzzle played on a square grid of coloured tiles. A click on any tile rotates the colour of that tile and every other tile sharing its row or column. This document gives a formal description of the problem (CSPLib prob027 [4]), works through a concrete example, states the key constraints, and formulates the problem as both an Integer Linear Program (ILP) and a Boolean Satisfiability (SAT) instance. Benchmark datasets are described and cited.

Contents

1 Introduction **2**

2 Problem Description **2**

3 Illustrative Example **3**

4 Key Constraints **3**

5 Integer Linear Programming (ILP) Model **4**
5.1 Decision variables . . . 4
5.2 Constraints . . . 4
5.3 Objective function . . . 4
5.4 Symmetry-breaking constraints (optional) . . . 5

6 SAT / Pseudo-Boolean Model **5**
6.1 Binary encoding of variables . . . 5
6.2 Linearised sum variables . . . 5
6.3 Modulo constraint in CNF . . . 5
6.4 Complexity summary . . . 6

7 SAT-Based Optimisation Approaches **6**
7.1 Non-incremental SAT (multiple independent calls) . . . 6
7.1.1 Binary search . . . 6
7.1.2 Linear search (top-down) . . . 7
7.2 Incremental SAT . . . 7
7.2.1 Assumption-based bound control . . . 7
7.3 MaxSAT and pseudo-Boolean optimisation . . . 7
7.3.1 Partial weighted MaxSAT . . . 7
7.3.2 Pseudo-Boolean optimisation (PBO) . . . 8
7.3.3 Weighted variant (binary encoding) . . . 8
7.4 Comparison of optimisation approaches . . . 8

8 Datasets and Benchmarks **8**
8.1 CSPLib instances 8
8.2 Instance parameters 8
8.3 Computational results from the literature 9

9 Conclusion **9**

1 Introduction

The Alien Tiles puzzle was originally available for play at `http://www.alientiles.com` [5]. It was listed as Problem 027 in *CSPLib*, a library of test problems for constraint solvers [4]. The puzzle belongs to the broader family of "Lights Out" games and is closely related to problems in coding theory over finite fields [3, 5].

The central combinatorial question is: *given an $N \times N$ grid with c colours and a goal state, what is the minimum number of clicks required to reach that state from the all-zero start state?* A harder meta-question asks for the *hardest* solvable instance, i.e., the goal state whose minimum-click solution is largest.

2 Problem Description

Definition 1 (Alien Tiles Puzzle)**.** *An instance of the Alien Tiles puzzle is specified by a triple (N, c, T) where*

- *$N \in \mathbb{N}$ is the grid size (the grid has $N \times N$ cells),*
- *$c \in \mathbb{N}$, $c \geq 2$ is the number of colours (equivalently, the modulus), and*
- *$T \in \mathbb{Z}_c^{N \times N}$ is the* target *(goal) state, where $\mathbb{Z}_c = \{0, 1, \ldots, c-1\}$.*

The start state *is the all-zero matrix $S = \mathbf{0}$.*

Click mechanics. A *click* on cell (i, j) adds 1 (modulo c) to the colour of every cell that shares row i or column j with (i, j). Formally, clicking (i, j) modifies the state matrix G as follows:

$$G'_{r,k} = \begin{cases} (G_{r,k} + 1) \bmod c & \text{if } r = i \text{ or } k = j, \\ G_{r,k} & \text{otherwise.} \end{cases}$$

Because clicks are *commutative* (the order does not matter), a solution is fully characterised by a *click matrix* $X = (x_{i,j}) \in \mathbb{Z}_c^{N \times N}$, where $x_{i,j}$ is the number of times cell (i, j) is clicked. Any c clicks on the same cell produce no net effect, so $0 \leq x_{i,j} \leq c - 1$.

Effect equation. After applying all clicks, the resulting colour of cell (r, k) is

$$G_{r,k} = \left(\sum_{j=1}^{N} x_{r,j} + \sum_{i=1}^{N} x_{i,k} - x_{r,k} \right) \bmod c. \tag{1}$$

Note that $x_{r,k}$ is counted twice (once in row r, once in column k), hence the subtraction of $x_{r,k}$.

Feasibility. A click matrix X is a *solution* if and only if

$$\forall\, r, k \in \{1, \ldots, N\} : \quad \left(\sum_{j=1}^{N} x_{r,j} + \sum_{i=1}^{N} x_{i,k} - x_{r,k} \right) \bmod c = t_{r,k}, \tag{2}$$

where $t_{r,k}$ is the target colour of cell (r, k).

3 Illustrative Example

Example 1 (4×4 grid, $c = 3$ colours)**.** *Consider $N = 4$ and $c = 3$ (colours 0, 1, 2 cycling as Red $\to$ Green $\to$ Blue $\to$ Red).*

Step 1: A single click. *Suppose we click cell $(1, 1)$ once. Every cell in row 1 and column 1 has its colour increased by 1 (mod 3). The resulting state is:*

	j=1	j=2	j=3	j=4
i=1	1	1	1	1
i=2	1	0	0	0
i=3	1	0	0	0
i=4	1	0	0	0

Cell $(1, 1)$ has value 1 because it lies in both row 1 and column 1, but the click only adds 1 once (not twice) to the clicked cell itself.

Step 2: Equivalence of solutions. *Consider clicking once in* every *cell of row 1 and twice in every cell of row 2. For cells in rows 3 and 4, each receives $4 \times 1 + 4 \times 2 = 12$ column-clicks from rows 1 and 2. Since $12 \bmod 3 = 0$, these cells are unchanged. For cells in rows 1 and 2, the row-clicks further accumulate to multiples of 3. The net effect on the entire grid is zero—this is a* null click pattern*. Any solution X can be replaced by $X' = X + \Delta$ (mod c) where Δ is such a null pattern, yielding an equivalent solution.*

Step 3: Hardest instance. *Among all solvable target states, we seek the one whose* minimum-click *representative (over all equivalent solutions) has the* largest *total click count. For $N = 4$, $c = 3$, the answer is 10 clicks, and there are 19 essentially different hardest instances (up to symmetry) [5].*

4 Key Constraints

The structure of the Alien Tiles problem gives rise to several important constraints and properties:

C1. Commutativity. The final state depends only on the *multiset* of clicks, not their order. Hence a solution is a matrix $X \in \mathbb{Z}_c^{N \times N}$, not a sequence.

C2. Modular arithmetic. All computations are performed in $\mathbb{Z}_c$. In particular, c clicks on any single cell produce no net effect, so $x_{i,j} \in \{0, 1, \ldots, c-1\}$.

C3. Effect equation. For each cell (r, k), the constraint

$$\Big(\sum_{j=1}^{N} x_{r,j} + \sum_{i=1}^{N} x_{i,k} - x_{r,k}\Big) \bmod c = t_{r,k}$$

must hold. This yields N^2 modular linear equations in N^2 unknowns.

C4. Equivalence classes. Two click matrices X and X' are *equivalent* if they produce the same goal state. Null patterns (click matrices that produce the all-zero effect) form a subgroup of $\mathbb{Z}_c^{N \times N}$.

C5. Symmetry. The problem exhibits a symmetry group of order $N! \times N! \times 2$ arising from:

- $N!$ row permutations,
- $N!$ column permutations, and
- reflection in the main diagonal (transposition).

These symmetries can be exploited through symmetry-breaking constraints to drastically reduce the search space [5].

C6. Objective (hardest-puzzle variant). Among all feasible click matrices for all solvable targets, find the target whose minimum-click representative maximises $\sum_{i,j} x_{i,j}$.

5 Integer Linear Programming (ILP) Model

We present the ILP formulation following Chlond [3].

5.1 Decision variables

- $x_{i,j} \in \{0, 1, \ldots, c-1\}$ for $i, j \in \{1, \ldots, N\}$: number of clicks on cell (i, j).
- $q_{r,k} \in \mathbb{Z}_{\geq 0}$: auxiliary integer variable used to linearise the modulo constraint for cell (r, k).

5.2 Constraints

The modular constraint (2) is linearised by introducing the auxiliary variables $q_{r,k}$:

$$\sum_{j=1}^{N} x_{r,j} + \sum_{i=1}^{N} x_{i,k} - x_{r,k} - c\, q_{r,k} = t_{r,k}, \qquad \forall\, r, k \in \{1, \ldots, N\}. \tag{3}$$

Variable bounds:

$$0 \leq x_{i,j} \leq c - 1, \qquad \forall\, i, j \in \{1, \ldots, N\}, \tag{4}$$

$$0 \leq q_{r,k} \leq \left\lfloor \frac{(2N-1)(c-1)}{c} \right\rfloor, \qquad \forall\, r, k \in \{1, \ldots, N\}. \tag{5}$$

The upper bound on $q_{r,k}$ follows from the fact that $\sum_j x_{r,j} + \sum_i x_{i,k} - x_{r,k} \leq (2N-1)(c-1)$.

5.3 Objective function

Variant 1: Solving a given target. If the goal is simply to find *any* feasible click matrix for a given target T, no objective is needed (feasibility problem).

Variant 2: Minimum total clicks for a given target.

$$\min \sum_{i=1}^{N} \sum_{j=1}^{N} x_{i,j} \quad \text{subject to (3)–(5).} \tag{6}$$

Variant 3: Hardest puzzle (max–min). This requires a bi-level formulation. One approach is to maximise over all solvable targets T:

$$\max_{T} \min_{X} \sum_{i,j} x_{i,j} \quad \text{s.t. } X \text{ solves } T. \tag{7}$$

Gent et al. solved this via constraint programming with symmetry breaking [5]; Chlond tackled it with ILP [3].

5.4 Symmetry-breaking constraints (optional)

To exploit the $N! \times N! \times 2$ symmetries, one can add:

$$\sum_{j=1}^{N} x_{1,j} \leq \sum_{j=1}^{N} x_{2,j} \leq \cdots \leq \sum_{j=1}^{N} x_{N,j}, \tag{8}$$

$$\sum_{i=1}^{N} x_{i,1} \leq \sum_{i=1}^{N} x_{i,2} \leq \cdots \leq \sum_{i=1}^{N} x_{i,N}, \tag{9}$$

$$x_{1,2} \leq x_{2,1}. \tag{10}$$

These break row permutation, column permutation, and diagonal reflection symmetries, respectively [5].

6 SAT / Pseudo-Boolean Model

We now encode the Alien Tiles problem as a propositional satisfiability (SAT) instance.

6.1 Binary encoding of variables

Each click variable $x_{i,j}$ takes values in $\{0, \ldots, c-1\}$ and is encoded using $b = \lceil \log_2 c \rceil$ Boolean variables:

$$x_{i,j} = \sum_{\ell=0}^{b-1} 2^{\ell}\, p_{i,j,\ell}, \qquad p_{i,j,\ell} \in \{0,1\}.$$

If c is not a power of 2, additional *at-most* clauses exclude the unused values $c, c+1, \ldots, 2^b - 1$.

6.2 Linearised sum variables

Define the sum for each cell (r,k) as:

$$\sigma_{r,k} = \sum_{j=1}^{N} x_{r,j} + \sum_{i=1}^{N} x_{i,k} - x_{r,k}.$$

This sum can range from 0 to $(2N-1)(c-1)$. Represent $\sigma_{r,k}$ in binary using $B = \lceil \log_2((2N-1)(c-1)+1) \rceil$ bits. The addition can be implemented via a *binary adder network* (e.g., ripple-carry or parallel-prefix adders) translated into CNF clauses using the Tseitin transformation.

6.3 Modulo constraint in CNF

The constraint $\sigma_{r,k} \bmod c = t_{r,k}$ can be encoded as:

$$\sigma_{r,k} - c\, q_{r,k} = t_{r,k},$$

where $q_{r,k}$ is an auxiliary integer variable encoded in binary. Alternatively, one can enumerate all valid assignments:

$$\sigma_{r,k} \in \{t_{r,k},\ t_{r,k}+c,\ t_{r,k}+2c,\ \ldots\},$$

and encode this as a disjunction of equality constraints, each expanded into CNF.

6.4 Complexity summary

Component	Boolean variables	Clauses
Click variables $p_{i,j,\ell}$	$N^2\lceil \log_2 c \rceil$	—
Sum auxiliary bits	$O(N^2 \log(Nc))$	$O(N^3 \log(Nc))$
Modulo constraints	$O(N^2 \log(Nc))$	$O(N^2 \cdot Nc/c)$
Value exclusion (if $c \neq 2^b$)	—	$O(N^2)$

7 SAT-Based Optimisation Approaches

The feasibility encoding of Section 6 determines whether a given target T is solvable. For the *minimisation* variant ($\min \sum_{i,j} x_{i,j}$), we must couple the feasibility formula with an objective. Three principal paradigms exist: non-incremental SAT, incremental SAT, and MaxSAT / pseudo-Boolean optimisation.

Unary expansion of the objective. To link the click values to a cardinality objective, we introduce *unit-contribution literals*. For each cell (i, j) and each $v \in \{1, \ldots, c-1\}$, define

$$u_{i,j,v} = 1 \iff x_{i,j} \geq v. \tag{11}$$

Then $\text{total}(X) = \sum_{i,j} \sum_{v=1}^{c-1} u_{i,j,v}$, and minimising total clicks becomes minimising the number of true u-literals. The channelling between $u_{i,j,v}$ and the click variables $d_{i,j,w}$ ("$x_{i,j} = w$") is:

$$u_{i,j,v} \iff \bigvee_{w=v}^{c-1} d_{i,j,w}.$$

This yields $n = N^2(c-1)$ unit literals in total.

7.1 Non-incremental SAT (multiple independent calls)

A sequence of *independent* SAT instances is solved, each with a different cardinality bound. A fresh solver is created for every call, so learned clauses are *not* reused.

7.1.1 Binary search

Search for the minimum K such that $\mathcal{F} \wedge \textsc{AtMost}(\{u_{i,j,v}\}, K)$ is satisfiable, where $\mathcal{F}$ denotes the feasibility formula:

```
ℓ ← 0;  h ← N²(c − 1);  X* ← None
while ℓ ≤ h do
    m ← ⌊(ℓ + h)/2⌋
    Build fresh CNF: F ∧ AtMost({u}, m)
    if Sat() then
        X* ← Extract();  h ← m − 1
    else  ℓ ← m + 1
    end if
end while
return X*                                   ▷ optimal in ⌈log₂(N²(c−1))⌉ SAT calls
```

The $\textsc{AtMost}$ constraint can be encoded via:

- **Sequential counter** [9]: $O(n \cdot K)$ auxiliary variables and clauses; compact with strong unit propagation.
- **Totalizer** [2]: $O(n \log n)$ variables, $O(n \log^2 n)$ clauses; often faster for large n.
- **Cardinality networks** [1]: sorting-network based; balances formula size and propagation strength.

7.1.2 Linear search (top-down)

Find any feasible solution X, record its total K, then solve with bound $K-1$, repeating until unsatisfiable. This needs $K_{\text{first}} - K^* + 1$ calls in the worst case and is effective when a good initial solution is found early.

7.2 Incremental SAT

A *single* solver instance is kept alive across iterations. Learned clauses are retained, making subsequent calls faster.

7.2.1 Assumption-based bound control

The cardinality encoding is built *once*. An *incremental totalizer* [6] produces output literals $o_1, o_2, \ldots, o_n$ where o_k is true if and only if at least k of the unit literals are true. To assert "at most K", the solver is called with the *assumption* $\neg o_{K+1}$:

```
solver ← IncrementalSolver(F)
Build totalizer on {u_{i,j,v}} → outputs o_1, ..., o_n
Add totalizer clauses to solver
ℓ ← 0;  h ← n;  X* ← None
while ℓ ≤ h do
    m ← ⌊(ℓ + h)/2⌋
    if solver.Solve(assumptions = {¬o_{m+1}}) then
        X* ← Extract();  h ← m − 1
    else  ℓ ← m + 1
    end if
end while
return X*
```

Key advantages over non-incremental search:

- The totalizer is built and added to the solver *once* (no re-encoding per call).
- Learned clauses from earlier iterations remain valid and accelerate later calls — typically 2–10× faster.
- Changing the bound only modifies the *assumptions*, not the clause database.

7.3 MaxSAT and pseudo-Boolean optimisation

Instead of iterating over bounds externally, the objective is encoded *inside* the formula and the solver handles optimisation internally.

7.3.1 Partial weighted MaxSAT

The formula is split into *hard* clauses (must be satisfied) and *soft* clauses (satisfy as many as possible):

- **Hard clauses:** all feasibility constraints $\mathcal{F}$ (exactly-one, running-sum, target assertion).
- **Soft clauses:** for each unit literal $u_{i,j,v}$, add the unit clause $(\neg u_{i,j,v})$ with weight 1.

Minimising total clicks is equivalent to *maximising* the number of satisfied soft clauses (i.e., maximising the literals $\neg u_{i,j,v}$ that are true).

Modern MaxSAT solvers employ several internal strategies:

- **Core-guided algorithms** (e.g., OLL [8], MSU3 [7]): extract unsatisfiable cores of soft clauses, add relaxation variables, and iterate. Particularly effective when the optimum is far from the worst case.
- **Linear SAT–UNSAT search:** internally equivalent to Section 7.1 but with learned-clause reuse.

- **Stratification:** partition soft clauses by weight and solve layer by layer; useful for weighted instances.

7.3.2 Pseudo-Boolean optimisation (PBO)

A PBO solver (e.g., RoundingSat) minimises a linear pseudo-Boolean objective directly:

$$\min \sum_{i,j} \sum_{v=1}^{c-1} u_{i,j,v} \qquad \text{subject to } \mathcal{F}. \tag{12}$$

PBO solvers use the *cutting-planes* proof system, which is strictly stronger than the resolution system underlying CDCL SAT solvers, and can therefore produce shorter proofs of optimality for certain structured instances.

7.3.3 Weighted variant (binary encoding)

If binary encoding is used for $x_{i,j}$ instead of one-hot, the objective is a *weighted* sum total $= \sum_{i,j} \sum_{\ell=0}^{b-1} 2^\ell \, p_{i,j,\ell}$. Each bit contributes a soft clause $(\neg p_{i,j,\ell})$ with weight 2^ℓ, yielding a *weighted* partial MaxSAT instance. Core-guided solvers handle weights via weight-aware core extraction (e.g., WPM1 with weight splitting).

7.4 Comparison of optimisation approaches

Table 1 summarises the key trade-offs.

Table 1: Comparison of SAT-based optimisation strategies for Variant 2.

Criterion	**Non-incremental**	**Incremental**	**MaxSAT / PBO**
SAT calls (external)	$O(\log(N^2c))$	$O(\log(N^2c))$	1
Learned-clause reuse	no	yes	yes
Cardinality encoder rebuilt	each call	once	once / internal
Implementation effort	low	medium	low
Optimality guarantee	binary search	binary search	solver-internal

8 Datasets and Benchmarks

8.1 CSPLib instances

The canonical dataset for the Alien Tiles problem is hosted at the CSPLib repository [4]:

`https://www.csplib.org/Problems/prob027/`

CSPLib provides:

- An Essence model for single-pass solving (`prob027-alientiles-singlepass.essence`).
- A multi-pass model using equivalence to determine minimality (`prob027-alientiles-multipass.zip`), based on the method of Gent, Linton, and Smith [5].
- Known results: for $N = 4$, $c = 3$, the hardest instance requires 10 clicks, and there are 19 distinct hardest instances up to symmetry [5].

8.2 Instance parameters

Table 2 summarises selected instance families.

Table 2: Selected instance families for the Alien Tiles problem.

N	c	# Variables	Known optimum	Source
4	3	16	10 clicks	Gent et al. [5]
N	$N-1$	N^2	open	CSPLib [4]
7	4	49	open	Chlond [3]

Generating test instances. For $N \times N$ with $c = N - 1$ colours, clicking 1 time in each cell of row 1 and $(N - 2)$ times in each cell of row 2 generates a null pattern, providing a convenient test instance family [4].

8.3 Computational results from the literature

Gent, Linton, and Smith [5] solved the N=4, c=3 instance using ILOG Solver with GAP-generated symmetry-breaking:

- Without symmetry breaking: 866 fails, 31 s to find optimum; 57 664 fails, 2 324 s to prove optimality.
- With symmetry breaking: 116 fails, 18.6 s; 499 fails, 56.6 s to prove optimality.

Chlond [3] formulated the 7×7, c=4 puzzle as an ILP and noted that transforming the entire grid to a single colour is particularly challenging, even with symmetry elimination.

9 Conclusion

The Alien Tiles problem is a rich combinatorial puzzle with connections to coding theory, linear algebra over finite fields, and symmetry exploitation. We have presented formal ILP and SAT encodings that can be used to solve instances and to search for the hardest puzzles. The CSPLib repository provides standard models and benchmarks for further research.

References

[1] Roberto Asín, Robert Nieuwenhuis, Albert Oliveras, and Enric Rodríguez-Carbonell. Cardinality networks: A theoretical and empirical study. In *Constraints*, volume 16, pages 195–221, 2011.

[2] Olivier Bailleux and Yacine Boufkhad. Efficient CNF encoding of boolean cardinality constraints. In *Proc. CP 2003*, volume 2833 of *LNCS*, pages 108–122. Springer, 2003.

[3] Martin J. Chlond. An alien IP. *INFORMS Transactions on Education*, 6(3):10–17, 2006.

[4] Ian P. Gent. CSPLib problem 027: Alien Tiles Problem. `https://www.csplib.org/Problems/prob027/`, 1999. Accessed: 2026-03-03.

[5] Ian P. Gent, Steve A. Linton, and Barbara M. Smith. Symmetry breaking in the alien tiles puzzle. *APES Research Report*, 22-2000, October 2000.

[6] Ruben Martins, Saurabh Joshi, Vasco Manquinho, and Inês Lynce. Incremental cardinality constraints for MaxSAT. In *Proc. CP 2014*, volume 8656 of *LNCS*, pages 531–548. Springer, 2014.

[7] Ruben Martins, Vasco Manquinho, and Inês Lynce. Open-WBO: A modular MaxSAT solver. In *Proc. SAT 2014*, volume 8561 of *LNCS*, pages 438–445. Springer, 2014.

[8] Antonio Morgado, Carmine Dodaro, and Joao Marques-Silva. Core-guided MaxSAT with soft cardinality constraints. In *Proc. CP 2014*, volume 8656 of *LNCS*, pages 564–573. Springer, 2014.

[9] Carsten Sinz. Towards an optimal CNF encoding of boolean cardinality constraints. In *Proc. CP 2005*, volume 3709 of *LNCS*, pages 827–831. Springer, 2005.